SAVAR Validation Pipeline for Causal Feature Discovery in Climate Foundation Models

Objective

Before applying causal feature discovery methods to GraphCast, validate the methodology on a system with known ground-truth causal structure.

The primary scientific question is:

> Are causally-important internal features also the features responsible for forecasting skill?

More precisely:

> Does the model's forecasting computation depend on the same causal pathways recovered through feature discovery?

SAVAR provides a controlled environment where the latent causal graph is known, allowing rigorous evaluation of:

1. Feature recovery
2. Causal discovery
3. Forecast interventions
4. Alignment between causal importance and forecast importance

---

Infrastructure

AWS Instance

Recommended:

- Instance: `g6e.xlarge`
- GPU: 1× NVIDIA L4 (24GB VRAM)
- CPU: 4 vCPU
- Memory: 16GB RAM

Alternative:

- `g6e.2xlarge` if data preprocessing becomes CPU-bound.

Avoid:

- A100
- H100
- Multi-GPU setups

The objective is methodological validation rather than large-scale training.

Storage

EBS

- Type: gp3
- Size: 200 GB

Expected usage:

Component	Storage
SAVAR data	<20 GB
Forecast checkpoints	<10 GB
SAE checkpoints	<50 GB
Analysis outputs	<20 GB
Miscellaneous	<20 GB

S3

Use S3 for long-term storage.

Suggested structure:

```
s3://savar-project/

    raw/
    processed/
    checkpoints/
    sae/
    graphs/
    results/
```

Workflow:

```
S3
 ↓
EBS local workspace
 ↓
Training / Analysis
 ↓
Results written back to S3
```

Do not mount S3 as a filesystem.

Phase 1: SAVAR Dataset

Goal

Construct:

```
Ground Truth Graph
        ↓
Latent Variables
        ↓
Observed Fields
        ↓
Forecasting Task
```

Forecast objective:

```
x_t → x_(t+1)
```

or

```
x_(t-k:t) → x_(t+1)
```

using a short temporal window.

Data Split

```
70% Training
15% Validation
15% Test
```

Store:

```json
{
    "observations": X,
    "latent_states": Z,
    "ground_truth_graph": G
}
```

Retaining latent variables is critical for evaluation.

Phase 2: Forecast Model

Recommended Architecture

Start with a small convolutional spatiotemporal forecaster.

Architecture

```
Past Frames
     ↓
3D Convolution
     ↓
Residual Block
     ↓
Residual Block
     ↓
Residual Block
     ↓
Prediction Head
     ↓
Next Frame Forecast
```

Target size:

```
2M–10M parameters
```

Reasons:

- Fast training
- Strong spatial inductive bias
- Easier activation analysis
- Simpler than transformers or GNNs

Do not begin with Graph Neural Networks.

Do not begin with Transformers.

The purpose is validating methodology.

Training Metrics

Primary:

- MSE
- RMSE

Secondary:

- Forecast correlation

Store all checkpoints.

Phase 3: Activation Collection

Choose one internal layer for mechanistic analysis.

Recommended:

```
Output of middle residual block
```

Collect activations:

```
activation[t, c, h, w]
```

for all training examples.

Save activation datasets separately.

Phase 4: Sparse Autoencoder (SAE)

Input

Internal activations from the forecast model.

Recommended format:

```
Patch-wise activations
```

rather than flattening entire spatial maps.

SAE Configuration

Target feature count:

```
1000–4000 features
```

Objective:

```
Reconstruction Loss
+
L1 Sparsity Penalty
```

Output:

```
f_1
f_2
...
f_n
```

These become candidate causal features.

Phase 5: Forecast Influence Analysis

For every SAE feature:

Intervention Types

Zero Ablation

```
f_i = 0
```

Mean Ablation

```
f_i = mean(f_i)
```

Scaling

```
f_i *= α
```

for multiple values of α.

Evaluation

Run the modified activations through the remaining model.

Measure:

Forecast Degradation

```
RMSE_delta
```

Physical Diagnostics

Domain-specific forecast metrics if available.

Output

Create:

```
forecast_importance[i]
```

for every feature.

This yields a ranking of forecast-driving features.

Phase 6: Causal Discovery

Construct feature time series.

For every timestep:

```
feature_vector_t
```

Result:

```
T × N_features
```

matrix.

Discovery Pipeline

Step 1

Mutual Information Screening

Remove weak candidate relationships.

Step 2

PCMCI / LPCMCI

Estimate causal graph structure.

Step 3

Orientation

Recover directed relationships where possible.

Output

Feature graph:

```
Feature A → Feature B
```

stored as:

```
feature_graph
```

Phase 7: Ground Truth Evaluation

This is the key advantage of SAVAR.

Map discovered features back to latent variables.

Mapping

For every feature:

```
corr(feature_i, latent_j)
```

or

```
MI(feature_i, latent_j)
```

Assign strongest correspondence.

Example:

```
Feature 14 ↔ Latent Variable 3
Feature 87 ↔ Latent Variable 7
```

Graph Evaluation

Recovered:

```
Feature14 → Feature87
```

Ground truth:

```
Latent3 → Latent7
```

Metrics:

Precision

Recall

F1

Structural Hamming Distance (SHD)

Orientation Accuracy

This should become a primary experimental result.

Phase 8: Main Hypothesis Test

The central experiment.

Question:

> Are causally-central features also forecast-important features?

Compute Causal Importance

Examples:

Out-Degree

```
out_degree(node)
```

Betweenness

PageRank

Ancestor Count

Descendant Count

Compare Against Forecast Importance

Using:

```
forecast_importance
```

from feature interventions.

Evaluate:

```
corr(
    causal_importance,
    forecast_importance
)
```

and rank-based metrics.

Expected Result

Determine whether:

```
Causally-important features
```

coincide with

```
Forecast-driving features
```

Phase 9: Edge Intervention Experiments

This is the most mechanistically meaningful experiment.

Rather than asking:

```
Does Feature A matter?
```

ask:

```
Does A → B matter?
```

Procedure

Normal execution:

```
A changes
  ↓
B changes
  ↓
Forecast
```

Intervention:

```
A changes
  ↓
Force B unchanged
  ↓
Forecast
```

Measure:

```
RMSE_delta
```

and diagnostic degradation.

Interpretation

Feature importance tells us:

```
Which features matter.
```

Edge interventions tell us:

```
Which causal pathways matter.
```

This provides stronger evidence that the discovered graph reflects forecasting computation.

Phase 10: Event Subspace Analysis

Only perform after the previous phases succeed.

Candidate events:

- Cyclones
- Heat Waves
- Atmospheric Rivers

Procedure

Collect activations during event occurrences.

Train:

- Linear probes
- PCA
- Sparse subspace methods

Estimate:

```
Effective dimensionality
```

of event representations.

Key Question

Rather than asking:

> Does a cyclone subspace exist?

Ask:

> Does the cyclone subspace correspond to a causal forecasting module?

This connects representation geometry to forecasting computation.

Transition to GraphCast

Move to GraphCast only after the following have been demonstrated on SAVAR:

1. SAE features recover latent variables.
2. Discovery pipeline recovers causal structure.
3. Forecast interventions identify forecast-driving features.
4. Causal centrality predicts forecast importance.
5. Edge interventions validate discovered pathways.

Only then apply the validated methodology to GraphCast.

Proposed Timeline

Week 1

- SAVAR setup
- CNN forecaster

Week 2

- Activation logging
- SAE training

Week 3

- Feature interventions
- Forecast influence ranking

Week 4

- Causal discovery pipeline

Week 5

- Ground-truth graph evaluation

Week 6

- Edge intervention experiments

Week 7+

- Event subspaces
- GraphCast scaling

Core Deliverable

The strongest scientific claim enabled by this pipeline is:

> Features that are causally central within a model's internal dynamics are disproportionately responsible for forecasting skill, and this relationship can be validated against known causal structure before being applied to frontier climate foundation models.